Postman API tesztelés sablon

Gyakorlati segédlet Laravel backend, Vue frontend és reszponzív webprojektek ellenőrzéséhez. Lépések, beállítások, példakérések, várt válaszok, státuszkódok, hibakeresés és automatikus tesztek.

1. Mi a Postman és mire használjuk?

A Postman egy API tesztelő eszköz. Arra való, hogy a backend végpontokat külön, frontend nélkül is ellenőrizni tudd. Így gyorsan kiderül, hogy a hiba a Laravel backendben, az adatbázisban, az API útvonalban vagy a Vue frontendben van-e.

Ha egy projekt elkészül, Postmannal végig lehet próbálni a legfontosabb műveleteket: lista lekérése, egy rekord lekérése, új rekord mentése, módosítás, törlés, validációs hiba ellenőrzése, jogosultság ellenőrzése.

> Lényeg: ha Postmanben jól működik az API, de Vue-ból nem, akkor általában frontend oldali URL, CORS, mezőnév vagy JavaScript hiba van. Ha Postmanben sem működik, akkor backend/adatbázis/route/controller hibát kell keresni.

2. Alap fogalmak

Fogalom	Mit jelent?	Példa
Request	Kérés, amit elküldesz a backendnek	GET /api/products
Response	Válasz, amit a backend visszaküld	JSON lista vagy hibaüzenet
Method	A kérés típusa	GET, POST, PUT, PATCH, DELETE
URL	Az API címe	http://127.0.0.1:8000/api/products
Headers	Kiegészítő információk a kéréshez	Accept: application/json
Body	Beküldött adat	JSON objektum POST kérésnél
Status code	HTTP válaszkód	200 OK, 201 Created, 422 Validation Error
Collection	Mentett kérések mappája	Projekt API tesztek
Environment	Változók gyűjteménye	{{base_url}}

3. Postman telepítés és első indítás

A Postman telepítése után érdemes létrehozni egy külön Collectiont minden projekthez. Például: Laravel API tesztek, Termék projekt API, Ingatlan projekt API. Így a kérések nem keverednek.

Ajánlott első beállítások

Lépés	Mit csinálj?
1	Nyisd meg a Postmant.
2	Hozz létre új Collectiont.
3	Adj neki beszédes nevet.
4	Hozz létre Environmentet, például Local Laravel néven.
5	Vegyél fel base_url változót.
6	Mentsd el a gyakori kéréseket a Collectionbe.

4. Laravel projekt indítása tesztelés előtt

Postman csak akkor tud válaszokat kapni, ha a Laravel backend fut, és az adatbázis is elérhető.

VS Code terminál vagy sima terminál

```
cd projekt_mappa

composer install
copy .env.example .env
php artisan key:generate

php artisan migrate
php artisan serve
```

> Várt eredmény: a Laravel kiírja, hogy fut például ezen a címen: http://127.0.0.1:8000. Ezt fogod Postmanben base_url-ként használni.

Ha Vue frontend is fut

```
# Első terminál: Laravel backend
php artisan serve

# Második terminál: Vue/Vite frontend
npm run dev
```

> Postmanhez a backend címe kell, nem a Vue frontend címe. API tesztnél általában http://127.0.0.1:8000/api/... címet használsz, nem a http://localhost:5173 címet.

5. Environment használata Postmanben

Az Environment azért hasznos, mert nem kell minden kérésben kézzel átírni az alap URL-t. Ha később más gépen vagy más porton fut a projekt, csak egy helyen módosítod.

Environment változók

Változó neve	Értéke	Mire való?
base_url	http://127.0.0.1:8000	Laravel backend alap címe
api_url	{{base_url}}/api	API prefix
token	ide jön a token	Bejelentkezéses API-khoz
product_id	1	Teszteléshez használt rekord id
category_id	1	Kapcsolt kategória id

URL használata változóval

```
{{api_url}}/products
{{api_url}}/products/{{product_id}}
{{api_url}}/categories
```

> Ha sárgán vagy pirosan jelzi a változót, akkor nincs kiválasztva Environment, vagy rossz a változó neve.

6. Kötelező headerek Laravel API teszthez

API kéréseknél érdemes mindig jelezni, hogy JSON választ vársz, és JSON adatot küldesz.

```
Accept: application/json
Content-Type: application/json
```

Bearer token esetén

```
Authorization: Bearer {{token}}
```

> Ha nem API route-ot használsz, hanem web.php route-ot, CSRF gond lehet. API teszteléshez általában routes/api.php végpontokat használj.

7. HTTP metódusok gyors értelmezése

Metódus	Mire való?	Várt sikeres kód
GET	Adatok lekérése	200 OK
POST	Új adat létrehozása	201 Created vagy 200 OK
PUT	Teljes rekord módosítása	200 OK
PATCH	Részleges módosítás	200 OK
DELETE	Rekord törlése	200 OK vagy 204 No Content

8. Teljes tesztelési sorrend egy CRUD API-hoz

Sorrend	Teszt	Miért fontos?
1	GET lista	Látszik, hogy az endpoint él-e
2	POST létrehozás	Mentés és validáció ellenőrzése
3	GET egy rekord	ID alapján visszakereshető-e
4	PUT/PATCH módosítás	Frissítés működik-e
5	DELETE törlés	Törlés működik-e
6	GET lista újra	Tényleg eltűnt vagy módosult-e
7	Hibás POST	Validáció megfelelő-e
8	Nem létező ID	404 kezelés működik-e

9. GET lista tesztelése

Elsőként mindig a lista lekérést próbáld. Ha ez sem működik, nincs értelme rögtön POST-tal kezdeni.

Postman beállítás

Mező	Érték
Method	GET
URL	{{api_url}}/products
Headers	Accept: application/json
Body	Nincs

Várt JSON válasz

```
[
  {
    "id": 1,
    "name": "Laptop",
    "description": "Használt üzleti laptop",
    "price": 180000,
    "active": true,
    "category_id": 1,
    "created_at": "2026-05-12T10:00:00.000000Z",
    "updated_at": "2026-05-12T10:00:00.000000Z"
  }
]
```

Automatikus Postman test

```
pm.test("Status code 200", function () {
    pm.response.to.have.status(200);
});

pm.test("A válasz tömb", function () {
    const json = pm.response.json();
    pm.expect(json).to.be.an("array");
});
```

Ha üres tömböt kapsz: [] - az nem feltétlen hiba. Azt jelenti, hogy az endpoint működik, csak nincs adat az adatbázisban.

10. GET egy rekord tesztelése

Postman beállítás

Mező	Érték
Method	GET
URL	{{api_url}}/products/1
Headers	Accept: application/json
Body	Nincs

Várt válasz

```
{
  "id": 1,
  "name": "Laptop",
  "description": "Használt üzleti laptop",
  "price": 180000,
  "active": true,
  "category_id": 1
}
```

Test

```
pm.test("Status code 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Van id mező", function () {
    const json = pm.response.json();
    pm.expect(json).to.have.property("id");
});
```

Ha 404-et kapsz, az adott ID nem létezik. Próbáld meg előbb a GET listával megnézni, milyen ID-k vannak.

11. POST új rekord létrehozása

POST kérésnél a Body részben raw JSON-t küldesz. Laravel oldalon a controller store metódusa dolgozza fel.

Postman beállítás

Mező	Érték
Method	POST
URL	{{api_url}}/products
Headers	Accept: application/json, Content-Type: application/json
Body	raw - JSON

Body példa

```
{
  "name": "Monitor",
  "description": "27 colos monitor",
  "price": 65000,
  "active": true,
  "category_id": 1
}
```

Várt válasz

```
{
  "id": 2,
  "name": "Monitor",
  "description": "27 colos monitor",
  "price": 65000,
  "active": true,
  "category_id": 1,
  "created_at": "2026-05-12T10:05:00.000000Z",
  "updated_at": "2026-05-12T10:05:00.000000Z"
}
```

Test

```
pm.test("Sikeres létrehozás", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

pm.test("A létrehozott elemnek van id-ja", function () {
    const json = pm.response.json();
    pm.expect(json).to.have.property("id");
    pm.environment.set("product_id", json.id);
});
```

A pm.environment.set('product_id', json.id) nagyon hasznos: a létrehozott rekord id-ját automatikusan elmenti, és a következő kéréseknél használható.

12. POST validációs hiba tesztelése

A jó backend nem enged be hibás adatot. Ezt külön tesztelni kell. Például hiányzó name, negatív price vagy nem létező category_id.

Hibás body példa

```
{
  "name": "",
  "description": "Hibás termék",
  "price": -500,
  "active": true,
  "category_id": 99999
}
```

Várt válasz

```
{
  "message": "The name field is required. (and 2 more errors)",
  "errors": {
    "name": [
      "The name field is required."
    ],
    "price": [
      "The price field must be at least 0."
    ],
    "category_id": [
      "The selected category id is invalid."
    ]
  }
}
```

Test

```
pm.test("Validációs hiba 422", function () {
    pm.response.to.have.status(422);
});

pm.test("Van errors objektum", function () {
    const json = pm.response.json();
    pm.expect(json).to.have.property("errors");
});
```

Ha nem 422 jön, hanem 500, akkor a backend nincs szépen validálva, hanem elszáll. Ilyenkor a controllerben validate szabályokat kell írni.

13. PUT vagy PATCH módosítás tesztelése

PUT/PATCH akkor kell, amikor egy meglévő rekordot módosítasz. A legtöbb Laravel API-ban működhet mindkettő, ha resource route van.

Postman beállítás

Mező	Érték
Method	PATCH
URL	{{api_url}}/products/{{product_id}}
Headers	Accept: application/json, Content-Type: application/json
Body	raw - JSON

Body példa

```
{
  "name": "Monitor módosítva",
  "price": 70000,
  "active": false
}
```

Test

```
pm.test("Sikeres módosítás", function () {
    pm.response.to.have.status(200);
});

pm.test("A név módosult", function () {
    const json = pm.response.json();
    pm.expect(json.name).to.eql("Monitor módosítva");
});
```

Ha 404 jön, rossz vagy nem létező product_id van az Environmentben. Ha 422 jön, a validáció nem fogadta el a beküldött adatot.

14. DELETE törlés tesztelése

Postman beállítás

Mező	Érték
Method	DELETE
URL	{{api_url}}/products/{{product_id}}
Headers	Accept: application/json
Body	Nincs

Várt válasz

```
{
  "message": "Sikeres törlés"
}
```

Test

```
pm.test("Sikeres törlés", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 204]);
});
```

Törlés ellenőrzése

```
# Következő kérés:
GET {{api_url}}/products/{{product_id}}

# Várt eredmény:
404 Not Found
```

A törlés után mindig kérdezd le ugyanazt az ID-t. Ha még mindig 200-at ad vissza, akkor nem törlődött a rekord.

15. Laravel resource controller és Postman megfeleltetés

Laravel metódus	HTTP	URL	Postman cél
index()	GET	/api/products	Összes rekord lekérése
store()	POST	/api/products	Új rekord mentése
show(Product $product)	GET	/api/products/{id}	Egy rekord lekérése
update(Request $request, Product $product)	PUT/PATCH	/api/products/{id}	Rekord módosítása
destroy(Product $product)	DELETE	/api/products/{id}	Rekord törlése

16. Kategória + termék kapcsolat tesztelése

Kapcsolt tábláknál először mindig a szülő adatot hozd létre. Például előbb category, utána product. Ha product category_id-t vár, de nincs ilyen kategória, validációs hiba lesz.

1. Kategória létrehozása

```
POST {{api_url}}/categories

{
  "name": "Elektronika"
}
```

Test - category_id mentése

```
pm.test("Kategória létrejött", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

const json = pm.response.json();
pm.environment.set("category_id", json.id);
```

2. Termék létrehozása a mentett kategóriával

```
POST {{api_url}}/products

{
  "name": "Billentyűzet",
  "description": "Mechanikus billentyűzet",
  "price": 32000,
  "active": true,
  "category_id": {{category_id}}
}
```

> Fontos: JSON body-ban a szám típusú environment változó idézőjel nélkül is használható. Szöveges változót idézőjelbe kell tenni.

17. Login és tokenes API tesztelés

Ha a backend bejelentkezést használ, először login kérést kell küldeni, majd a kapott tokent el kell menteni Environment változóba.

Login példa

```
POST {{api_url}}/login

{
  "email": "teszt@example.com",
  "password": "password"
}
```

Várt válasz példa

```
{
```

```
  "user": {
    "id": 1,
    "name": "Teszt Elek",
    "email": "teszt@example.com"
  },
  "token": "1|abcdef123456..."
}
```

Token automatikus mentése

```
pm.test("Login sikeres", function () {
    pm.response.to.have.status(200);
});

const json = pm.response.json();
pm.environment.set("token", json.token);
```

Védett kérésnél header

```
Authorization: Bearer {{token}}
Accept: application/json
```

> Ha 401 Unauthorized jön, nincs token, rossz token, lejárt token, vagy a backend route auth middleware mögött van és nincs megfelelő jogosultság.

18. Vue frontend hibakeresés Postman alapján

Ha Laravel API Postmanben jól működik, de Vue-ban nem, akkor így keresd a hibát:

Ellenőrzés	Mit nézz?
URL	Vue ugyanazt az endpointot hívja, amit Postmanben teszteltél?
Method	POST helyett véletlen GET? PATCH helyett PUT?
Headers	Content-Type és Accept helyes?
Body	A mezőnevek pontosan egyeznek? name vagy nev? price vagy ar?
CORS	Böngésző Console ír-e CORS hibát?
Response	Vue-ban console.log(await response.json()) mit mutat?
Validation	422 esetén a backend errors objektumát jeleníted meg?

Vue fetch példa Postman alapján

```
async function ment() {
  const response = await fetch('http://127.0.0.1:8000/api/products', {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      name: form.name,
      description: form.description,
      price: form.price,
      active: form.active,
      category_id: form.category_id
    })
  })

  const data = await response.json()
  console.log(data)
}
```

> Ha Postmanben működik ugyanez a body, de Vue-ban nem, akkor általában form mezőnév, URL, CORS vagy JSON.stringify hiányzik.

19. Collection szervezése

Egy jól rendszerezett Collection sok időt spórol. Érdemes mappákba rendezni az endpointokat.

Mappa	Kérések
Auth	Register, Login, Logout, Me
Categories	List, Create, Show, Update, Delete
Products	List, Create, Show, Update, Delete
Validation tests	Hibás create, hibás update
Edge cases	Nem létező ID, üres lista, jogosulatlan kérés

Elnevezési minta

```
Products - List
Products - Create valid
Products - Create invalid
Products - Show
Products - Update
Products - Delete
Products - Show deleted item
```

20. Collection Runner használata

A Collection Runnerrel több mentett kérést egymás után le tudsz futtatni. Ez akkor hasznos, ha a teljes API működését gyorsan akarod ellenőrizni.

Ajánlott sorrend Runnerhez

Sorrend	Kérés
1	Categories - Create
2	Products - Create valid
3	Products - List
4	Products - Show
5	Products - Update
6	Products - Delete
7	Products - Show deleted item
8	Products - Create invalid

A sorrend fontos. Ha előbb futtatod a termék létrehozását, mint a kategóriát, és category_id kell, akkor a termék létrehozás elhasalhat.

21. Automatikus tesztek Postmanben

A Tests fülön JavaScript alapú ellenőrzéseket írhatsz. Ezek megmondják, hogy az adott kérés rendben lefutott-e.

Státuszkód ellenőrzés

```
pm.test("Status 200", function () {
    pm.response.to.have.status(200);
});
```

JSON mező ellenőrzés

```
pm.test("Van name mező", function () {
    const json = pm.response.json();
    pm.expect(json).to.have.property("name");
});
```

Array ellenőrzés

```
pm.test("A válasz lista", function () {
    const json = pm.response.json();
    pm.expect(json).to.be.an("array");
});
```

Érték ellenőrzés

```
pm.test("Az ár nagyobb mint 0", function () {
    const json = pm.response.json();
    pm.expect(json.price).to.be.above(0);
});
```

Environment változó mentés

```
const json = pm.response.json();
pm.environment.set("product_id", json.id);
```

Response idő ellenőrzés

```
pm.test("Gyors válasz", function () {
    pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

22. Pre-request Script - automatikus adatok

A Pre-request Script a kérés elküldése előtt fut le. Hasznos például véletlen név vagy időbélyeg generálásához.

```
const random = Math.floor(Math.random() * 100000);

pm.environment.set("random_name", "Teszt termék " + random);
pm.environment.set("random_price", Math.floor(Math.random() * 100000) + 1000);
```

Body használat változókkal

```
{
  "name": "{{random_name}}",
  "description": "Automatikusan generált tesztadat",
  "price": {{random_price}},
  "active": true,
  "category_id": {{category_id}}
}
```

> Ez azért jó, mert többször is lefuttathatod ugyanazt a create tesztet anélkül, hogy ugyanazt a nevet küldenéd be.

23. Várt státuszkódok gyors táblázata

Kód	Jelentés	Mikor jó?
200 OK	Sikeres kérés	GET, PUT, PATCH, DELETE válasz
201 Created	Sikeres létrehozás	POST után

Kód	Jelentés	Mikor jó?
204 No Content	Sikeres, nincs válasz body	DELETE után gyakori
400 Bad Request	Rossz kérés	Hibás formátum
401 Unauthorized	Nincs bejelentkezés/token	Védett API token nélkül
403 Forbidden	Nincs jogosultság	Be van jelentkezve, de nem teheti meg
404 Not Found	Nem található	Rossz URL vagy nem létező ID
405 Method Not Allowed	Rossz HTTP metódus	GET helyett POST kellene
419 Page Expired	CSRF probléma web route-nál	API helyett web route használat
422 Unprocessable Entity	Validációs hiba	Hiányzó vagy rossz mező
500 Server Error	Backend elszállt	Controller, model, DB vagy kódhiba

24. Gyakori hibák és javítások

Hiba	Valószínű ok	Javítás
Could not get response	Laravel szerver nem fut	php artisan serve
404 Not Found	Rossz URL vagy nincs route	php artisan route:list
405 Method Not Allowed	Rossz metódus	GET/POST/PATCH/DELETE ellenőrzés
422 Validation Error	Rossz body vagy hiányzó mező	JSON mezőnevek javítása
500 Server Error	Backend kódhiba	storage/logs/laravel.log megnézése
419 Page Expired	CSRF	API route használata, Accept JSON
401 Unauthorized	Hiányzó token	Authorization Bearer token beállítás
SQLSTATE error	Adatbázis/migration gond	.env, migrate, tábla ellenőrzés
CORS hiba Vue-ban	Böngésző blokkol	Laravel CORS/config ellenőrzése
HTML jön JSON helyett	Rossz route vagy nincs Accept header	Accept: application/json

25. Laravel route ellenőrzés terminálban

Ha Postmanben 404 vagy 405 hibát kapsz, nézd meg, valóban létezik-e az útvonal.

```
php artisan route:list
```

Mit keress?

```
GET|HEAD   api/products
POST       api/products
GET|HEAD   api/products/{product}
PUT|PATCH  api/products/{product}
DELETE     api/products/{product}
```

> Ha az endpoint nincs a listában, akkor a routes/api.php fájlban nincs jól felvéve, vagy cache probléma van.

Route cache törlés

```
php artisan route:clear
php artisan config:clear
php artisan cache:clear
```

26. JSON body helyes formátuma

Postmanben Body -> raw -> JSON típust válassz. A JSON nagyon szigorú: dupla idézőjel kell, nincs pontosvessző, a true/false kisbetűs.

Helyes

```
{
  "name": "Laptop",
  "price": 180000,
  "active": true
}
```

Hibás

```
{
  name: "Laptop",
  price: "180000";
  active: True
}
```

> Ha a JSON hibás, a Postman vagy a backend nem tudja feldolgozni. Mindig figyelj a dupla idézőjelekre és a vesszőkre.

27. Fájl feltöltés tesztelése

Ha a backend képfeltöltést vagy dokumentum feltöltést kér, akkor nem raw JSON-t használsz, hanem form-data body-t.

Postman beállítás

Mező	Érték
Method	POST
URL	{{api_url}}/products
Body	form-data
image	File típus
name	Text típus
price	Text típus

Laravel controller példa

```
$request->validate([
    'name' => 'required|string|max:255',
    'price' => 'required|integer',
    'image' => 'nullable|image|max:2048'
]);

$path = null;

if ($request->hasFile('image')) {
    $path = $request->file('image')->store('products', 'public');
}
```

> Fájlfeltöltésnél ne állítsd kézzel Content-Type: application/json-ra, mert form-data kell.

28. Teljes termék API tesztcsomag példa

1. Categories - Create

```
POST {{api_url}}/categories

Headers:
Accept: application/json
Content-Type: application/json

Body:
{
  "name": "Teszt kategória"
}

Tests:
pm.test("Kategória létrejött", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

const json = pm.response.json();
pm.environment.set("category_id", json.id);
```

2. Products - Create

```
POST {{api_url}}/products

Headers:
Accept: application/json
Content-Type: application/json

Body:
{
  "name": "Teszt termék",
  "description": "Postmanből létrehozva",
  "price": 12000,
  "active": true,
  "category_id": {{category_id}}
}

Tests:
pm.test("Termék létrejött", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

const json = pm.response.json();
pm.environment.set("product_id", json.id);
```

3. Products - List

```
GET {{api_url}}/products

Tests:
pm.test("Lista lekérve", function () {
    pm.response.to.have.status(200);
});

pm.test("JSON lista", function () {
    pm.expect(pm.response.json()).to.be.an("array");
});
```

4. Products - Show

```
GET {{api_url}}/products/{{product_id}}

Tests:
pm.test("Egy termék lekérve", function () {
    pm.response.to.have.status(200);
});

pm.test("Az id egyezik", function () {
    const json = pm.response.json();
    pm.expect(json.id).to.eql(Number(pm.environment.get("product_id")));
});
```

5. Products - Update

```
PATCH {{api_url}}/products/{{product_id}}

Body:
{
  "name": "Módosított teszt termék",
  "price": 15000
}
```

```
Tests:
pm.test("Termék módosítva", function () {
    pm.response.to.have.status(200);
});

pm.test("Név módosult", function () {
    const json = pm.response.json();
    pm.expect(json.name).to.eql("Módosított teszt termék");
});
```

6. Products - Delete

```
DELETE {{api_url}}/products/{{product_id}}

Tests:
pm.test("Termék törölve", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 204]);
});
```

7. Products - Show deleted

```
GET {{api_url}}/products/{{product_id}}

Tests:
pm.test("Törölt elem már nem elérhető", function () {
    pm.response.to.have.status(404);
});
```

29. Ha a feladat más témájú, mit kell átírni?

Ha a feladat ezt kéri	Mit írj át a sablonban?
Ingatlanok	products -> ingatlanok/properties, name -> cim, price -> ar
Túrák	products -> tours, name -> leiras, price -> tav
Autók	products -> cars, name -> marka, price -> ar
Kategóriák	categories endpoint marad vagy saját név: tipusok
Aktív/inaktív mező	active boolean maradhat
Dátum mező	JSON-ban például: "date": "2026-05-12"
Kapcsolt tábla	category_id helyett driver_id, user_id, type_id stb.
Képfeltöltés	raw JSON helyett form-data

30. Ellenőrzőlista beadás/kész projekt előtt

Kész?	Ellenőrzés
[ ]	Laravel szerver fut
[ ]	Adatbázis beállítás jó az .env fájlban
[ ]	Migrationök lefutottak
[ ]	php artisan route:list mutatja az API route-okat
[ ]	GET lista működik
[ ]	POST létrehozás működik
[ ]	Validációs hiba 422-vel jön
[ ]	GET egy rekord működik
[ ]	PATCH/PUT módosítás működik
[ ]	DELETE törlés működik

Kész?	Ellenőrzés
[ ]	Nem létező ID 404-et ad
[ ]	Vue ugyanazokat az endpointokat használja
[ ]	Ha token kell, Authorization Bearer működik
[ ]	Postman Collection mentve van

31. Gyors diagnosztikai döntési fa

Tünet	Következő lépés
Postman nem kap választ	php artisan serve fut-e? Jó portot használsz?
404	php artisan route:list, URL ellenőrzés
405	HTTP metódus ellenőrzés
422	Body mezőnevek és validate szabályok ellenőrzése
500	Laravel log megnyitása
Vue hibázik, Postman jó	Frontend URL, CORS, body, headers ellenőrzés
Postman jó, adatbázisban mégsem látod	Másik adatbázisba csatlakozik az .env
Tokenes kérés nem működik	Authorization header, token mentése, auth middleware

32. Minimum Postman tudás, amit érdemes biztosan tudni

Tudás	Miért kell?
Method kiválasztása	GET/POST/PATCH/DELETE nem felcserélhető
URL pontos megadása	Egy karakter hiba is 404 lehet
Headers használata	Laravel JSON választ adjon
Body raw JSON	POST/PATCH adatok küldéséhez
Environment változók	Gyorsabb teszteléshez
Tests fül	Automatikus OK/hiba visszajelzéshez
Collection Runner	Teljes folyamat gyors ellenőrzéséhez
Status code értelmezés	Azonnal tudd, milyen típusú a hiba

33. Rövid mintaválaszok értelmezése

Minden rendben

```
Status: 200 OK
Body: JSON adat vagy lista
Jelentés: Az endpoint működik.
```

Új adat létrejött

```
Status: 201 Created
Body: Az új rekord adatai id-val
Jelentés: A POST mentés sikeres.
```

Validációs hiba

```
Status: 422 Unprocessable Entity
Body: errors objektum
Jelentés: A backend jól védekezik, a beküldött adat hibás.
```

Nem található

```
Status: 404 Not Found
Jelentés: Rossz URL vagy nem létező rekord ID.
```

Szerverhiba

```
Status: 500 Internal Server Error
Jelentés: Laravel oldali hiba, logot kell nézni.
```

34. Záró sablon: minden új API-nál ezt kövesd

```
1. Indítsd el a backendet:
   php artisan serve

2. Nézd meg a route-okat:
   php artisan route:list

3. Postmanben állítsd be:
   base_url = http://127.0.0.1:8000
   api_url = {{base_url}}/api

4. Teszteld:
   GET lista
   POST létrehozás
   GET egy rekord
   PATCH módosítás
   DELETE törlés
   Hibás POST validációra

5. Mentsd a fontos ID-kat Environmentbe:
   pm.environment.set("product_id", json.id);

6. Ha minden zöld:
   frontend összekötés jöhet.
```